



TAREA #1

UNIDAD 4

CARRERA:

INGENERIA DE SOFTWARE

ASIGNATURA:

APLICACIONES WEB

GRUPO:

2

TEMA DE LA UNIDAD:

DESPLIEGUE Y PUBLICACIÓN DE APLICACIONES

ACTIVIDAD:

DESPLIEGUE DEL PROYECTO DE LA UNIDAD 3 (MERCAPP) EN MONGODB ATLAS, RAILWAY, NETLIFY Y PUBLICACIÓN DE MICROSITIO EN GITHUB PAGES

ESTUDIANTE:

AIDAN DAMIAN FLORES GUEVARA

DOCENTE:

ING. ING. JOSE JAIME

Objetivo General

Desplegar en Internet el proyecto desarrollado en la Unidad 3 (SPA con Vue 3 + API propia), realizando el aprovisionamiento de la base de datos en MongoDB Atlas, el despliegue del back-end en Railway, el despliegue del front-end en Netlify y la publicación de un micrositio del proyecto en GitHub Pages, garantizando HTTPS, CORS correctamente configurado, uso de variables de entorno y artefactos de build optimizados.

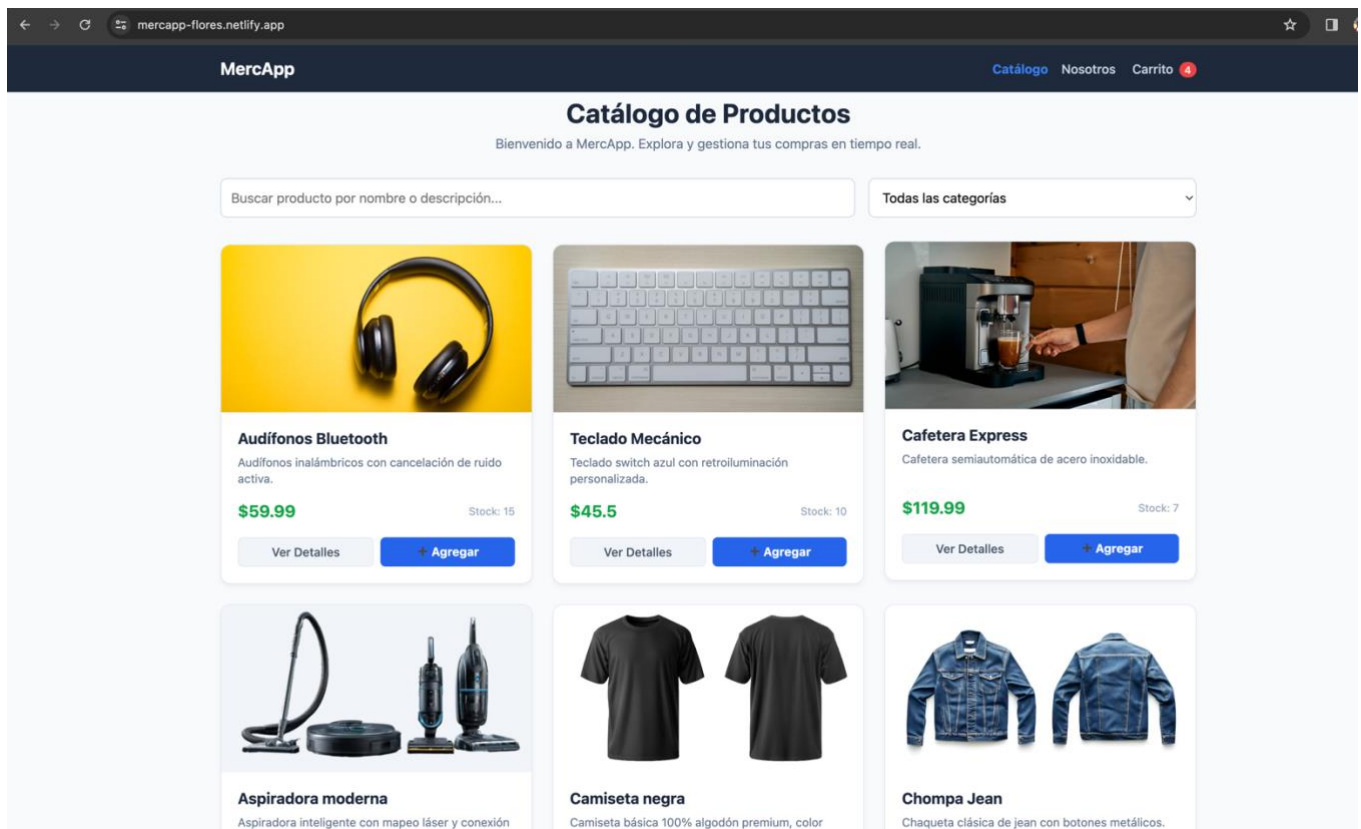
Introducción

MercApp es un sistema de gestión de inventario, esta diseñado para optimizar el control de los productos en tiempo real. Históricamente el seguimiento manual de los inventarios conlleva errores humanos como la pérdida de información y la falta de visibilidad sobre los niveles del stock. MercApp soluciona esto ya que proporciona una plataforma web que conecta una base de datos enfocada a la interfaz de usuario intuitiva garantizando de esta manera que los datos sean precisos, actualizados y consultables desde cualquier ubicación.

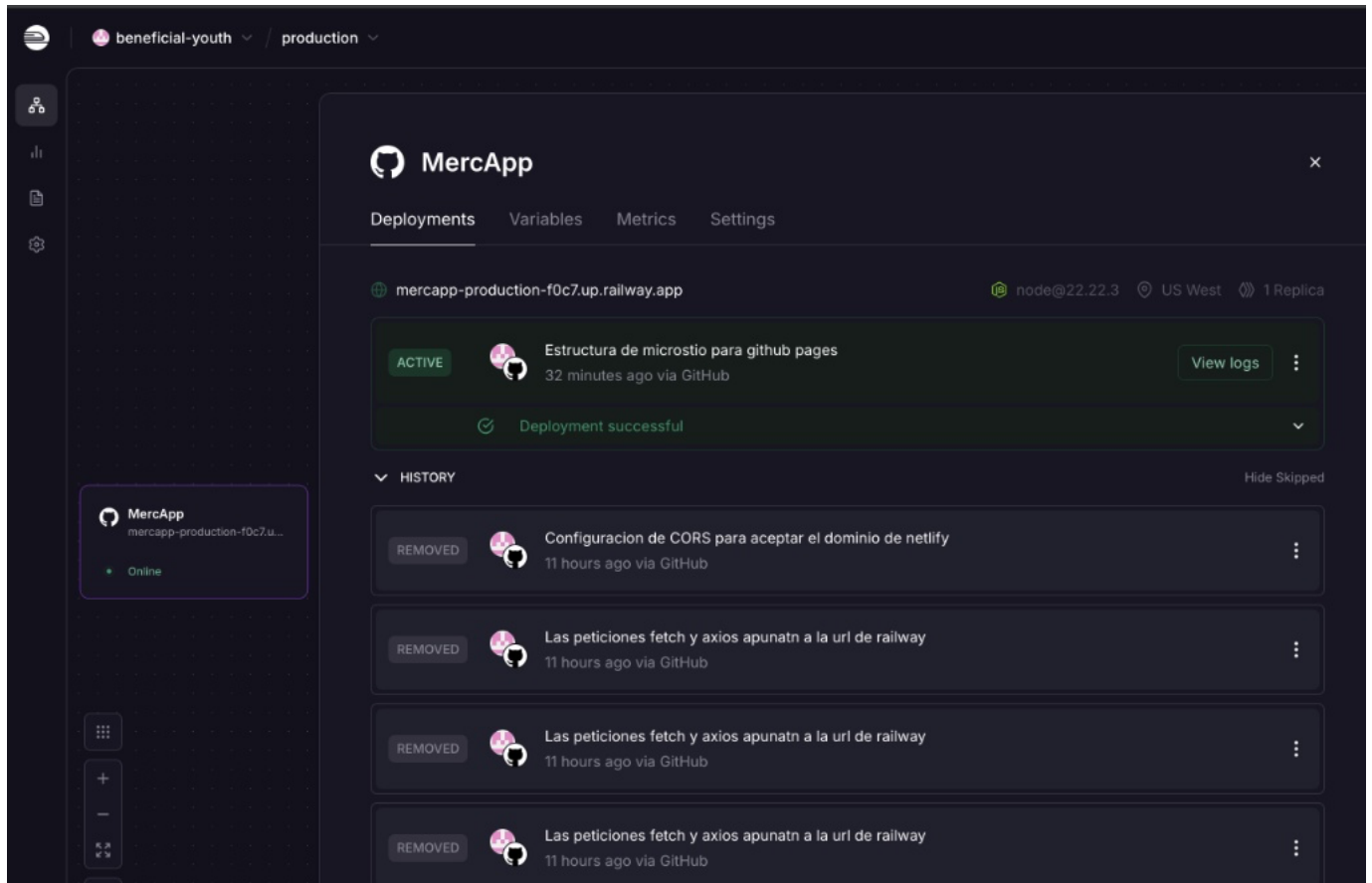
Como funciona?

La pagina emplea un frontend de Vue el cual envía solicitudes HTTP al backend que es Express. El backend es el intermediario ya que consulta a la base de datos no SQL de MongoDB para facilitar la tarea de la entrega de datos en el formato JSON al cliente.

Interfaz de usuario integrada consumiendo servicios reales



Servidor de MercApp corriendo en el puerto 3000



Validación de que la API está operativa y conectada a la base de datos



Resolución de Problemas

Durante e desarrollo se encontró varios problemas de políticas de seguridad que impedían y limitaban el acceso de la comunicación entre el dominio de Netlify y el servidor de Sailway. Para que no pase esto en el archivo server.js del backend se implemento explícitamente los orígenes permitidos de esta manera garantiza que la conexión sea segura y estable.

INSTRUCCIONES PARA LA REALIZACIÓN DE LA ACTIVIDAD DE LA UNIDAD 4

1. Crear un repositorio en GitHub y trabajar de forma síncrona con el proyecto
2. Generar un archivo README.md indicando los datos del estudiante, las funcionalidades implementadas e instrucciones básicas de uso
3. Coloca la URL de su repositorio de GitHub en un archivo readme.txt dentro de la carpeta principal del proyecto
4. Una vez concluido su desarrollo, **comprimir la carpeta en un archivo ZIP y enviar al AVAC** con el siguiente formato “Actividad4-Flores-Aidan.zip”.

Rúbrica:

Criterios	Bajo (0%)	Regular (30%)	Medio (60%)	Alto (100%)	Sub-puntaje máximo
1) Despliegue y operatividad end-to-end (Railway + Netlify + Atlas + Pages)	API/SPA no accesibles; errores de CORS que bloquean; BD no conectada; rutas SPA rotas; URLs públicas ausentes. 0	Al menos 1 componente publicado pero con fallos que impiden el flujo (p. ej., SPA carga pero API falla o CORS interrumpe); sin health check; GitHub Pages ausente. 1.8	API en Railway y SPA en Netlify accesibles con CRUD parcial; CORS resuelto parcialmente; rutas SPA con incidencias menores; GitHub Pages publicado pero desactualizado o sin enlaces completos. 3.6	CRUD completo, búsqueda/filtros; routing SPA correcto (fallback configurado), API con health, CORS restringido al dominio de Netlify, Atlas operativo con pruebas; GitHub Pages con enlaces a Netlify/Railway y descripción breve; todo bajo HTTPS. 6	6
2) Seguridad y configuración de entorno (secretos, CORS, HTTPS, acceso a BD)	Datos sensibles en el repositorio; sin .env.example; HTTP; allowlist de Atlas inapropiada; sin cabeceras de seguridad. 0	Variables de entorno parcialmente; .env.example ausente; CORS permisivo ("*") con credenciales; allowlist amplia sin justificación; sin verificación TLS/Compass. 0.9	Variables y secretos bien gestionados (no versionados) y .env.example; CORS restringido parcialmente; HTTPS activo; evidencia básica de verificación TLS/Compass; allowlist temporal documentada. 1.8	Gestión robusta: secretos fuera del repo, .env.example, uso de npm ci; CORS estrictamente por dominio, HTTPS y cabeceras (p. ej., Helmet); rate-limiting/sanitización básicos; allowlist mínima y evidencias de pruebas TLS/Compass. 3	3.0

3) Observabilidad, rendimiento y documentación de despliegue (logs, build, Pages, README)	Sin revisión de logs; build no optimizado (modo dev); sin Pages; README insuficiente. 0	Revisión ocasional de logs; build de producción dudoso; Pages con información mínima; README parcial y sin enlaces. 0.9	Logs consultados y capturas de pruebas; build de producción (artefactos dist) y SPA funcional; GitHub Pages con arquitectura y endpoints; README con instalación/ejecución y enlaces básicos. 1.8	Evidencias sistematizadas (capturas/links); uso de logs para diagnóstico; mejoras de rendimiento verificadas (p. ej., revisión con Lighthouse); SPA con redirecciones/headers cuando aplica; Pages con diagrama, endpoints, versiones; README completo con enlaces a Netlify/Railway/Pages y checklist final. 3	3.0
Totales					12

